# Autonomous Business Intelligence & Agentic Automation Platform

Complete Project Blueprint — End to End

Authored by: Jameel Ahmad | AI/ML Engineer

Stack: Python · FastAPI · Streamlit · Ollama · n8n · Docker · VS Code

May 2026

# 1. Problem Statement

Modern businesses drown in fragmented tools — CRMs, email clients, research dashboards, support desks, reporting suites — each siloed, requiring manual hand-offs and context switching. There is no unified intelligent layer that autonomously orchestrates tasks, understands documents, remembers context, and communicates across channels (WhatsApp, Slack, Email) without human intervention. NexusOS solves this by providing a single AI-powered platform that connects every business function through intelligent agents, automated workflows, and a conversational interface.

# 2. Tools & Technologies

| Layer | Technology / Tool | Purpose |
|---|---|---|
| Frontend UI | Streamlit | Interactive web dashboard & chat interface |
| Backend API | FastAPI | REST API, async endpoints, agent orchestration |
| LLM Engine | Ollama (Llama3, Phi3, Mistral) | Local inference — no GPU needed, 40GB RAM |
| Workflow Automation | n8n (self-hosted) | Email, calendar, WhatsApp, Slack, Discord, scheduling |
| Vector Store | ChromaDB / FAISS | RAG embeddings & semantic document search |
| Embeddings | sentence-transformers (MiniLM) | Text vectorization for RAG pipeline |
| Database | SQLite → PostgreSQL | User data, CRM records, logs, memory |
| Long-term Memory | Mem0 / custom Redis store | Persistent agent context across sessions |
| Agent Framework | LangChain / LangGraph | Multi-agent orchestration & tool routing |
| Browser Automation | Playwright / Selenium | Web scraping & competitor analysis |
| Speech / Voice | Whisper (OpenAI, local) | Meeting transcription & voice input |
| File Processing | pdfplumber, openpyxl, pandas | CSV / PDF / Excel ingestion & analysis |
| Containerization | Docker + Docker Compose | Microservice deployment & isolation |
| IDE | VS Code + Python ext. | Development environment |
| Notifications | n8n webhooks + SMTP | Real-time alerts & monitoring |

# 3. AI / ML Coverage

| Domain | What You Will Cover |
|---|---|
| NLP | Text classification, NER, intent detection, summarization, sentiment analysis via LLMs |
| Generative AI | Prompt engineering, RAG pipelines, instruction-tuned local LLMs (Llama3/Mistral/Phi3) |

| | |
|---|---|
| Agentic AI | ReAct agents, tool-use agents, multi-agent systems using LangGraph |
| Deep Learning | Transformer-based embeddings (MiniLM), Whisper ASR for transcription |
| ML Pipelines | Data ingestion, preprocessing, vector indexing, retrieval-augmented generation |
| Recommendation | Priority scoring, lead ranking using rule-based + LLM hybrid approach |
| Memory Systems | Short-term (session) + long-term (vector DB) context retention for agents |

## 4. Architecture Layers & Flow

| | | |
|---|---|---|
| **L1** | **Presentation Layer** | Streamlit UI — Chat, Dashboard, File Upload, Reports |
| **L2** | **API Gateway Layer** | FastAPI — REST endpoints, auth, request routing |
| **L3** | **Agent Orchestration Layer** | LangChain / LangGraph — Multi-agent task planning & tool dispatch |
| **L4** | **LLM Inference Layer** | Ollama — Llama3 / Mistral / Phi3 local model serving |
| **L5** | **RAG / Memory Layer** | ChromaDB + Mem0 — Vector search, embeddings, long-term memory |
| **L6** | **Automation Layer** | n8n — Workflow triggers, email, calendar, Slack, WhatsApp |
| **L7** | **Data & Storage Layer** | PostgreSQL + Redis — Structured data, CRM, sessions, cache |
| **L8** | **Infrastructure Layer** | Docker Compose — Service isolation, networking, scaling |

*Flow: User Input (Voice/Text) → L1 Streamlit → L2 FastAPI → L3 Agent Router → L4 Ollama LLM → L5 RAG/Memory → L6 n8n Automation → L7 DB → Response back to L1*

# 5. 20 Development Tasks — Step by Step

## TASK 01 Project Setup & Docker Environment

*Purpose: Establish the full development environment so all services run consistently from day one.*

### Step 1 — Install Prerequisites

- Install Python 3.11, VS Code, Docker Desktop, Ollama on Windows. Verify each with version commands.

### Step 2 — Pull Ollama Models

- Run: ollama pull llama3 && ollama pull mistral && ollama pull phi3 to download local LLMs.

### Step 3 — Create Project Structure

- Create folders: nexusos/backend, nexusos/frontend, nexusos/agents, nexusos/workflows, nexusos/data.

### Step 4 — Write docker-compose.yml

- Define services: fastapi, streamlit, postgres, redis, chromadb, n8n — link via internal Docker network.

### Step 5 — Test All Services Start

- Run docker compose up and verify all containers are healthy via docker ps and browser checks.

---

## TASK 02 FastAPI Backend Skeleton

*Purpose: Build the core API backbone that all other modules will plug into.*

### Step 1 — Initialize FastAPI App

- Create main.py with FastAPI(), add CORS middleware, define /health endpoint.

### Step 2 — Add Authentication

- Implement JWT-based auth with /login and /register endpoints using python-jose.

### Step 3 — Create Router Structure

- Add APIRouter for: /agents, /chat, /documents, /crm, /reports — each in separate files.

### Step 4 — Connect PostgreSQL

- Use SQLAlchemy async with asyncpg, define User, Session, Task models and run Alembic migrations.

### Step 5 — Add Request Logging

- Middleware to log every request with timestamp, user_id, endpoint to a logs table.

---

## TASK 03 Streamlit Frontend Dashboard

*Purpose: Create the main user interface where all platform features are accessible.*

### Step 1 — Setup Multi-page App

- Use st.navigation() to create pages: Chat, Dashboard, CRM, Reports, Settings.

### Step 2 — Build Chat UI

- Chat page with st.chat_input, st.chat_message, message history stored in st.session_state.

### Step 3 — API Integration

- Use httpx to call FastAPI endpoints from Streamlit — abstract into api_client.py helper.

### Step 4 — Add Authentication UI

- Login / register form on first load, store JWT token in st.session_state for subsequent calls.

### Step 5 — Dashboard Metrics

- Use st.metric, st.plotly_chart to show KPIs: active tasks, leads, emails processed today.

---

## TASK 04 Ollama LLM Integration

*Purpose: Connect local LLMs as the core intelligence engine with proper prompt management.*

### Step 1 — Ollama Client Wrapper

- Create llm_client.py wrapping ollama Python library with model selection (llama3/mistral/phi3).

### Step 2 — Prompt Template System

- Build PromptTemplate class — system prompt + user message + history injection for each use case.

### Step 3 — Streaming Responses

- Implement async streaming via ollama.chat(stream=True), yield chunks to FastAPI StreamingResponse.

### Step 4 — Model Router

- Auto-select model by task: phi3 for quick tasks, llama3 for reasoning, mistral for code generation.

### Step 5 — Response Caching

- Cache identical prompts in Redis with 1-hour TTL to save compute time on repeated queries.

---

## TASK 05 RAG Pipeline — Document Understanding

*Purpose: Enable the platform to ingest, index, and intelligently answer questions from business documents.*

### Step 1 — Document Ingestion API

■ POST /documents/upload endpoint accepting PDF, DOCX, CSV, TXT — extract text via pdfplumber/python-docx.

### Step 2 — Chunking Strategy

■ Split documents into 512-token chunks with 50-token overlap using LangChain TextSplitter.

### Step 3 — Embed & Store in ChromaDB

■ Generate embeddings with sentence-transformers/all-MiniLM-L6-v2, upsert into ChromaDB collection.

### Step 4 — Semantic Search Endpoint

■ POST /documents/query — embed user query, retrieve top-5 chunks, build context for LLM.

### Step 5 — RAG Chain Assembly

■ LangChain RetrievalQA chain: query → retriever → LLM → answer with source citations.

---

## TASK 06 Long-Term Memory System

*Purpose: Give agents persistent memory so they remember user preferences and past interactions.*

### Step 1 — Memory Schema Design

■ Create memory table in PostgreSQL: user_id, memory_key, content, embedding, created_at.

### Step 2 — Mem0 Integration

■ Install mem0ai, configure with local Ollama embeddings and PostgreSQL storage backend.

### Step 3 — Memory Write Hook

■ After every chat session, extract key facts (name, preferences, decisions) and save to memory.

### Step 4 — Memory Retrieval

■ On new chat, query top-5 relevant memories via cosine similarity and inject into system prompt.

### Step 5 — Memory Management UI

■ Streamlit settings page — list user memories, allow manual delete or edit.

---

## TASK 07 Multi-Agent Orchestration

*Purpose: Build autonomous agents that can plan, use tools, and collaborate to complete complex tasks.*

### Step 1 — Define Agent Base Class

■ Abstract Agent with: name, tools[], system_prompt, run(task) method using LangGraph StateGraph.

### Step 2 — Build Core Agents

■ Create: ResearchAgent, EmailAgent, CRMAgent, ReportAgent, SchedulerAgent as subclasses.

### Step 3 — Tool Registry

■ Register tools (web_search, send_email, query_db, scrape_url) with JSON schema descriptions for LLM.

### Step 4 — LangGraph Task Graph

■ Define StateGraph nodes per agent — edges based on task type routing decided by supervisor LLM.

### Step 5 — Agent Execution API

■ POST /agents/run with task description — returns task_id, stream status updates via SSE.

---

TASK 08

## Email Automation via n8n

*Purpose: Automate email reading, classification, drafting, and sending through n8n workflows.*

### Step 1 — Setup n8n Self-Hosted

■ Run n8n via Docker, configure SMTP credentials (Gmail/Outlook) in n8n credentials store.

### Step 2 — Email Trigger Workflow

■ n8n workflow: IMAP trigger → HTTP Request to FastAPI /agents/email → classify & draft reply.

### Step 3 — Auto-Reply Agent

■ EmailAgent reads email body, determines intent (support/sales/inquiry), drafts reply via LLM.

### Step 4 — Send Email Endpoint

■ FastAPI POST /email/send — n8n calls this to send drafted reply via SMTP after approval.

### Step 5 — Email Dashboard

■ Streamlit page showing inbox summary, auto-drafted replies pending human approval before send.

## TASK 09 Calendar Automation via n8n

*Purpose: Automate meeting scheduling, reminders, and calendar event management.*

### Step 1 — Google Calendar OAuth in n8n

- Configure Google Calendar node in n8n with OAuth2 — test by fetching today's events.

### Step 2 — Meeting Scheduler Workflow

- n8n workflow: receive meeting request → check calendar availability → propose 3 time slots via email.

### Step 3 — Reminder Automation

- Cron trigger in n8n: 30 min before event → send Slack/WhatsApp reminder with agenda summary.

### Step 4 — Calendar API Endpoint

- FastAPI /calendar/events GET endpoint — proxies n8n data to Streamlit calendar view.

### Step 5 — Meeting Brief Generator

- Pre-meeting agent: pull attendees from CRM, past interactions from memory, generate 1-page brief.

---

## TASK 10 Meeting Transcription & Summarization

*Purpose: Automatically transcribe audio/video meetings and extract action items using Whisper + LLM.*

### Step 1 — Whisper Integration

- Install openai-whisper (local), create POST /transcribe endpoint accepting MP3/MP4/WAV files.

### Step 2 — Async Transcription Job

- Use FastAPI BackgroundTasks to run Whisper transcription — return job_id immediately.

### Step 3 — Transcription Polling

- GET /transcribe/{job_id} — returns status (pending/done) and transcript text when complete.

### Step 4 — LLM Summarization

- Pass transcript to LLM with prompt: extract summary, key decisions, and action items as JSON.

### Step 5 — Transcript Storage & Search

- Store transcript + summary in PostgreSQL, embed in ChromaDB for future RAG queries.

---

## TASK 11 CRM & Lead Management

*Purpose: Build an intelligent CRM that tracks leads, scores them, and recommends follow-up actions.*

### Step 1 — CRM Database Schema

- Tables: Lead (name, email, company, source, score, status), Interaction (lead_id, type, notes, date).

### Step 2 — Lead Ingestion API

- POST /crm/leads — accept leads from n8n (web forms, email parsing, CSV upload).

### Step 3 — AI Lead Scoring

- LLM + rule-based hybrid: score 0-100 based on company size, interaction frequency, email sentiment.

### Step 4 — Follow-up Recommender

- After each interaction, LLM recommends next action: call, send proposal, demo, nurture email.

### Step 5 — CRM Streamlit Page

- Kanban-style board (st.columns) showing leads by stage: New → Contacted → Qualified → Closed.

---

TASK 12

## AI Research & Competitor Analysis

*Purpose: Automate web research and competitor monitoring using browser automation + LLM synthesis.*

### Step 1 — Playwright Setup

- Install playwright, create async scraper with stealth settings to avoid bot detection.

### Step 2 — Research Agent Tool

- Tool: web_search(query) → fetch top 5 URLs → scrape text → chunk → summarize per source.

### Step 3 — Competitor Monitor Workflow

- n8n scheduled job: daily scrape competitor websites, detect changes, alert via Slack if significant.

### Step 4 — Research Report Generator

- ResearchAgent synthesizes findings into structured report: overview, strengths, gaps, opportunities.

### Step 5 — Research API Endpoint

- POST /research/run with topic — async job, result stored in DB and downloadable as PDF.

---

## TASK 13 AI Customer Support & Call Assistance

*Purpose: Provide real-time AI assistance for customer support queries via chat and call transcription.*

### Step 1 — Support Chat Endpoint

- POST /support/chat — takes customer message, retrieves relevant docs from RAG, generates answer.

### Step 2 — Ticket Classification

- NLP classifier (LLM zero-shot) to tag tickets: billing, technical, general, urgent — auto-route.

### Step 3 — Live Call Assist

- Stream Whisper transcription in real-time, surface relevant KB articles in agent dashboard.

### Step 4 — Escalation Logic

- If LLM confidence < threshold or keyword triggers detected, flag for human escalation in Streamlit.

### Step 5 — Support Analytics

- Dashboard: ticket volume, avg resolution time, top issues, satisfaction trend (weekly LLM sentiment scan).

---

## TASK 14 CSV / Excel / PDF Business Data Analysis

*Purpose: Allow users to upload business data files and ask natural language questions about them.*

### Step 1 — File Upload Endpoint

- POST /analyze/upload — accept CSV/XLSX/PDF, parse with pandas/pdfplumber, store in temp session.

### Step 2 — Data Profiling

- Auto-generate schema summary: column names, dtypes, null counts, basic stats — shown in Streamlit.

### Step 3 — NL-to-Analysis Agent

- User asks 'show top 10 customers by revenue' → LLM generates pandas code → execute safely → return result.

### Step 4 — Chart Generation

- Use plotly to auto-generate charts based on LLM-suggested visualization type for the query.

### Step 5 — Export Results

- Download analyzed data + charts as PDF report via reportlab — POST /analyze/export.

---

## TASK 15 Report Generation & Business Analytics

*Purpose: Automatically generate formatted business reports from data and agent outputs.*

### Step 1 — Report Template System

- Define report types: Daily Summary, Lead Report, Research Report, Support Report with jinja2 templates.

### Step 2 — Data Aggregation Agent

- ReportAgent pulls data from PostgreSQL, ChromaDB, and n8n logs to compile report content.

### Step 3 — LLM Narrative Generation

- LLM writes executive summary and key insights section in natural language from raw data.

### Step 4 — PDF Export via ReportLab

- Generate professional PDF with company logo, tables, charts, page numbers using reportlab platypus.

### Step 5 — Scheduled Report Delivery

- n8n cron: generate report → save PDF → email to configured recipients every Monday 8AM.

---

## TASK 16 WhatsApp / Slack / Discord Integration

*Purpose: Connect the platform to messaging channels for notifications, commands, and bot interactions.*

### Step 1 — Slack Bot via n8n

- n8n Slack trigger — receive slash commands (/nexus research ) → call FastAPI → reply in thread.

### Step 2 — WhatsApp via n8n + Twilio

- n8n WhatsApp node (Twilio) — receive messages → process via agents → reply with results.

### Step 3 — Discord Bot Webhook

- n8n webhook → Discord bot posts daily digests, alerts, and report summaries to configured channels.

### Step 4 — Notification Service

- FastAPI /notify endpoint — any agent can trigger notification via POST with channel + message.

### Step 5 — Channel Preference Settings

- User settings page: choose preferred notification channel per alert type (email/slack/whatsapp).

---

## TASK 17 Real-Time Notifications & Monitoring

*Purpose: Monitor system health, agent tasks, and business events with live alerts.*

**Step 1 — Task Status Tracking**

- Every agent task updates status in Redis (pending/running/done/failed) — poll via GET /tasks/{id}.

**Step 2 — SSE Live Feed**

- FastAPI Server-Sent Events endpoint /events/stream — Streamlit listens and shows live task updates.

**Step 3 — Health Check Dashboard**

- Streamlit monitoring page: ping all services (Ollama, ChromaDB, Postgres, n8n) every 30s, show status.

**Step 4 — Alert Rules Engine**

- n8n workflow monitors: if lead count drops, support queue > 50, disk > 80% → send alert.

**Step 5 — System Metrics Logger**

- Background task logs CPU, RAM, active agents to metrics table every 5 minutes — plotted in dashboard.

---

## TASK 18 Secure Multi-User SaaS Architecture

*Purpose: Make the platform usable by multiple users/organizations with proper isolation and security.*

**Step 1 — Multi-Tenant Schema**

- Add organization_id to all DB tables — all queries scoped by org. Middleware injects org from JWT.

**Step 2 — Role-Based Access Control**

- Roles: Admin, Manager, User — each has different API permissions enforced via FastAPI dependencies.

**Step 3 — API Key Management**

- Organizations can generate API keys (POST /auth/apikey) for programmatic n8n/external access.

**Step 4 — Data Isolation**

- Separate ChromaDB collections and Redis namespaces per organization — no cross-tenant data leakage.

**Step 5 — Audit Logging**

- Log all data access events (who accessed what, when) to audit_log table — viewable by Admin only.

---

## TASK 19 Browser Automation & Web Scraping

*Purpose: Automate repetitive browser tasks and extract structured data from web sources.*

**Step 1 — Playwright Async Setup**

- Create browser_agent.py with async Playwright context, handle cookies, JS rendering, timeouts.

**Step 2 — Structured Scraper Tool**

- Tool: scrape(url, selectors_dict) → returns clean JSON — used by ResearchAgent and CompetitorAgent.

**Step 3 — Form Automation**

- Playwright fills and submits web forms (e.g., job portals, lead forms) based on LLM-generated inputs.

**Step 4 — Anti-Detection Measures**

- Rotate user-agents, add random delays, use playwright-stealth plugin to avoid blocking.

**Step 5 — Scraping Job Queue**

- Celery or FastAPI BackgroundTasks queue for bulk scraping jobs — results stored and searchable.

---

## TASK 20 Final Integration, Testing & Deployment

*Purpose: Wire all modules together, run end-to-end tests, and deploy the complete platform.*

### Step 1 — Integration Testing

- Write pytest tests for each FastAPI endpoint — mock LLM calls, use test DB, run in CI.

### Step 2 — End-to-End Workflow Test

- Manually test: upload doc → ask question → agent researches → sends email → calendar scheduled.

### Step 3 — Performance Tuning

- Profile slow endpoints with py-spy, optimize DB queries with EXPLAIN ANALYZE, add Redis caching.

### Step 4 — Docker Production Build

- Write production docker-compose.prod.yml with env secrets, volume mounts, restart policies.

### Step 5 — Documentation & Handoff

- Auto-generate API docs via FastAPI /docs, write README with setup steps, record Loom walkthrough video.

---

# 6. Recommended Build Order

| | |
|---|---|
| **Phase 1 — Foundation (Week 1-2)** | Tasks 1, 2, 3, 4 — Get Docker, FastAPI, Streamlit, Ollama running together. |
| **Phase 2 — Intelligence (Week 3-4)** | Tasks 5, 6, 7 — RAG pipeline, memory system, multi-agent framework. |
| **Phase 3 — Automation (Week 5-6)** | Tasks 8, 9, 10 — Email, calendar, meeting transcription via n8n. |
| **Phase 4 — Business Features (Week 7-9)** | Tasks 11, 12, 13, 14, 15 — CRM, research, support, data analysis, reports. |
| **Phase 5 — Integrations (Week 10-11)** | Tasks 16, 17, 18, 19 — Messaging channels, notifications, security, browser automation. |
| **Phase 6 — Shipping (Week 12)** | Task 20 — Full integration testing, performance tuning, production deployment. |